

ITERATOR, GENERATOR, DECORATOR

1. ITERATORS

Definition:

An Iterator is an object that helps us iterate over an iterable type one value at a time. It allows fetching only one value when required.

Iterator Protocol:

1. `__iter__()`: Initializes the iterator and returns itself.
2. `__next__()`: Returns next value, raises `StopIteration` if no values.

Iterator vs Looping:

- Iterator: Explicit control, custom logic, maintains state.
- Looping: Implicit control, simple syntax, fixed logic.

```
class numbers:
    def __init__(self):
        self.num = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.num <= 10:
            res = self.num
            self.num += 1
            return res
        else:
            raise StopIteration
```

```
n1 = numbers()
print(n1.__next__()) # Output: 1
```

2. GENERATORS

Definition:

A Generator is a function that yields values one at a time using 'yield'.

Key Points:

- Uses 'yield' instead of return.
- Remembers previous state.
- Memory efficient.

Function vs Generator:

- Function: Executes completely and returns value.
- Generator: Pauses and resumes execution.

```
def usn_generator(college, yoj, stream):
    num = 1
    while True:
        yield f"{college}{yoj}{stream}{num}"
        num += 1
```

```
usn = usn_generator('BMS', '24', 'CS')
print(next(usn)) # Output: BMS24CS1
```

3. DECORATORS

Definition:

A Decorator modifies or extends function behavior without changing its code.

Rules:

1. Must be nested function.
2. Takes function as argument.
3. Returns function.

Types:

- In-built: @staticmethod, @abstractmethod
- Custom decorators

```
def calculate_total_price(param):
    def wrapper(qty, price):
        total = qty * price
        if total < 500:
            total += 60
        elif total < 1000:
            total += 30
        print(f"Total price for {param.__name__}: {total}")
        return param(qty, price)
    return wrapper

@calculate_total_price
def pizza(qty, price):
    print("Order details processed.")

pizza(2, 254)
```